Lab 03- Docker

Fichiers fournis : app.py, requirements.txt, Dockerfile (incomplet), test_api.py (incomplet)

Rendu : Un seul fichier compressé (zip, rar, …) nommé selon la classe (INFO 1 ou 2) **INFO1_FamilyName_FirstName.txt,** contenant un rapport (Word ou PDF) + les fichiers produits, déposés à la fin de la séance.

Exercice 1 : Déploiement de Base

1. Compléter le fichier Dockerfile afin de créer une image docker contenant l'API app.py
2. Construire l'image Docker
3. Exécuter le conteneur
4. Vérifier que l'API est accessible
5. Consulter les logs du conteneur

Livrable : Capture d'écran d'une réponse API réussie

Exercice 2 : Tests de l'API

Tester toutes les opérations CRUD :

1. Lister tous les livres
2. Ajouter 2 nouveaux livres
3. Mettre à jour les informations d'un livre
4. Supprimer un livre
5. Vérifier les modifications
6. Compléter le fichier test_api.py

Livrable : Documenter chaque requête et réponse

Exercice 3 : Gestion des Conteneurs

1. Arrêter le conteneur
2. Le redémarrer
3. Consulter les logs
4. Exécuter une commande dans le conteneur pour vérifier la version de Python

Livrable : Commandes utilisées et leurs sorties

Exercice 4 : Amélioration de la sécurité

1. Pour des mesures de sécurité, il est déconseillé d'exécuter un serveur en mode root. Créez un Dockerfile ‘Dockerfile-apiuser’ qui permet de créer un utilisateur apiuser via lequel sera exécuté le serveur Flask
2. Construire l'image Docker
3. Exécuter le conteneur
4. Vérifier que le serveur Flask est lancé par l’utilisateur apiuser

Livrable : Commandes utilisées et leurs sorties

Exercice 5 : Gestion des logs

Par défaut, Flask écrit ses logs sur la sortie standard : ils sont lus par docker logs mais disparaissent avec le conteneur. Vous allez les écrire dans un fichier stocké dans un volume Docker.

1. Créer le volume : docker volume create log-volume, puis l’inspecter (docker volume inspect).
2. Faire en sorte que le serveur écrive ses logs dans un fichier (ex. /var/log/api/api.log). Deux approches possibles : (a) configurer un FileHandler dans app.py ; (b) rediriger la sortie du serveur dans le CMD. Choisissez-en une et justifiez votre choix.
3. Créer Dockerfile-logs (basé sur Dockerfile-apiuser) : créer le répertoire de logs et donner la propriété à apiuser.
4. Lancer le conteneur en montant le volume : -v log-volume:/var/log/api. Générer quelques requêtes (succès et erreurs).
5. Arrêter et supprimer le conteneur.
6. Vérifier que les logs sont toujours dans le volume, à l’aide d’un conteneur jetable (par exemple docker run --rm -v log-volume:/data alpine cat /data/api.log).
7. Relancer un nouveau conteneur avec le même volume, générer une requête et vérifier que l’ancien contenu est conservé et complété.

Livrable : Commandes utilisées et leurs sorties